

SATQUERY AI

P5 — Atharv

COMPLETE ENGINEERING PRD

Frontend / Geospatial UX Engineer

Deadline: 20 September 2026 • Six developers in parallel • Contract-first microservices

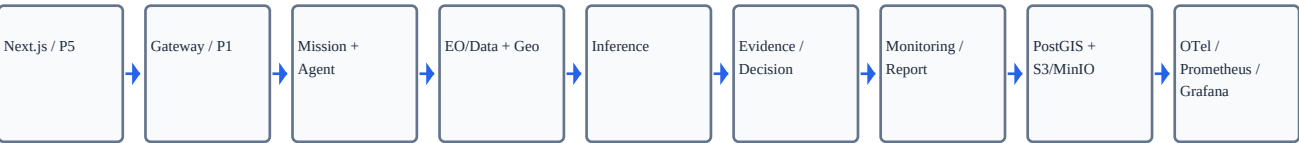
This is an execution document, not an API-only specification. It defines ownership, repository locations, modules, issues, dependencies, tests, security, observability, daily execution and a ready-to-copy coding-agent prompt.

1. Role Mission

Field	Implementation specification
Primary owner	Next.js, MapLibre, mission console, evidence/timeline/report UX
Backup/review	API integration
Architecture reviewer	P1
QA/DevOps	P6
Success condition	Owned subsystem is independently runnable, contract-conformant, observable, secure and integrated into the flagship flood mission.
Non-ownership	Other teams' private tables, internal implementation details, credentials and undocumented contract changes.

2. System Position

P5 system position



Communication: browser→Gateway uses HTTPS/WebSocket; service-to-service uses versioned APIs/events; long-running EO/ML/report/monitoring work is asynchronous; storage and schemas have explicit ownership.

3. Repository / Folder Ownership

Folder	Purpose	Must not contain
apps/web/app/	Owned implementation, tests or docs	Unrelated service logic / credentials
apps/web/components/	Owned implementation, tests or docs	Unrelated service logic / credentials
apps/web/lib/api/	Owned implementation, tests or docs	Unrelated service logic / credentials
apps/web/lib/geo/	Owned implementation, tests or docs	Unrelated service logic / credentials
apps/web/e2e/	Owned implementation, tests or docs	Unrelated service logic / credentials
packages/contracts/generated/ts/	Owned implementation, tests or docs	Unrelated service logic / credentials

4. Technology Stack

Field	Implementation specification
Stack	TypeScript/Next.js/React/Tailwind/shadcn/MapLibre/deck.gl/Turf/generated OpenAPI client/Playwright
Why	Matches the supplied SatQuery architecture and the six-person playbook; additions are implementation recommendations, not product requirements.
Serialization	OpenAPI/JSON Schema; canonical DTOs/events.
Testing	pytest and contract/integration tests; frontend uses browser E2E where applicable.
Observability	OpenTelemetry + structured logs + Prometheus/Grafana.
Containerization	Docker; Kubernetes remains future/post-SIH.

5. Engineering Rules

- All internal APIs use /api/v1; breaking changes require versioning and P1 review.
- Canonical ErrorResponse: code, message, details[], retryable, trace_id.
- Propagate trace_id, mission_id, run_id, job_id, organization_id where applicable.
- organization_id comes from verified auth context, never a request body.
- No cross-service private DB reads.
- Long-running EO/ML/report/monitoring work returns 202 + job_id.
- Results retain dataset_id/observation_ids/model_version/processing_version as applicable.
- Idempotency-Key is used for duplicate-sensitive writes.

- Remote calls use bounded timeouts/retries/backoff; no retry storms.
- Never log secrets, auth headers or raw sensitive inputs.

6. Complete Issue Register

ID	Title	Pri	Type	Depends	Contact
P5-01	Next.js application shell and design system	P0	Frontend	P1-01	P1
P5-02	Generated API client integration	P0	Integration	P1-01,P1-02	P4
P5-03	Mission console query and planning UX	P0	Feature	P1-01,P1-02	P2
P5-04	Mission status and run timeline	P0	Feature	P5-03	P1
P5-05	MapLibre map workspace	P0	Geo UX	P5-04	P4
P5-06	AOI draw/edit and validation UX	P0	Geo UX	P5-05	P2
P5-07	Observation and before/after comparison	P0	Geo UX	P5-06,P1-02	P1
P5-08	Change polygons, masks and confidence visualization	P0	Geo UX	P5-07,P1-02	P4
P5-09	WHY? evidence panel	P0	Feature	P5-08,P1-02	P2
P5-10	Sensor arbitration and disagreement UI	P1	Feature	P5-09,P1-02	P1
P5-11	Confidence/uncertainty and human-in-loop UX	P0	Safety UX	P5-10	P4
P5-12	Mission memory and temporal history	P0	Feature	P5-11	P2
P5-13	Persistent monitoring mission screen	P0	Feature	P5-12	P1
P5-14	Report generation and report view	P0	Feature	P5-13	P4
P5-15	Admin, security and accessibility UX	P1	Security/UX	P5-14,P4-05	P2
P5-16	Frontend performance and error boundaries	P1	Performance	P5-15,P4-05	P1
P5-17	E2E UX hardening and deterministic demo mode	P0	Release	P5-16,P4-05	P4

P5-01 — Next.js application shell and design system

Field	Implementation specification
Priority	P0
Type	Frontend
Owner	P5
Reviewer	P1
Cross-service contact	P1
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement next.js application shell and design system as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for next.js application shell and design system. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_next_js_application_shell_and_design_system_valid()', 'test_next_js_application_shell_and_design_system_invalid_input()']
Integration tests	['test_p5_01_service_boundary()']
Contract tests	['test_p5_01_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, next.js application shell and design system completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-02
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run next.js application shell and design system against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-02 — Generated API client integration

Field	Implementation specification
Priority	P0
Type	Integration
Owner	P5
Reviewer	P1
Cross-service contact	P4
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement generated api client integration as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for generated api client integration. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_generated_api_client_integration_valid()', 'test_generated_api_client_integration_invalid_input()']
Integration tests	['test_p5_02_service_boundary()']
Contract tests	['test_p5_02_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, generated api client integration completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-03
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run generated api client integration against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-03 — Mission console query and planning UX

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P5
Reviewer	P1
Cross-service contact	P2
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement mission console query and planning ux as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for mission console query and planning ux. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_mission_console_query_and_planning_ux_valid()', 'test_mission_console_query_and_planning_ux_invalid_input()']
Integration tests	['test_p5_03_service_boundary()']
Contract tests	['test_p5_03_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, mission console query and planning ux completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-04
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run mission console query and planning ux against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-04 — Mission status and run timeline

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P5
Reviewer	P1
Cross-service contact	P1
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement mission status and run timeline as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for mission status and run timeline. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_mission_status_and_run_timeline_valid()', 'test_mission_status_and_run_timeline_invalid_input()']
Integration tests	['test_p5_04_service_boundary()']
Contract tests	['test_p5_04_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, mission status and run timeline completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-03
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-05
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run mission status and run timeline against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-05 — MapLibre map workspace

Field	Implementation specification
Priority	P0
Type	Geo UX
Owner	P5
Reviewer	P1
Cross-service contact	P4
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement maplibre map workspace as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for maplibre map workspace. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_maplibre_map_workspace_valid()', 'test_maplibre_map_workspace_invalid_input()']
Integration tests	['test_p5_05_service_boundary()']
Contract tests	['test_p5_05_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, maplibre map workspace completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-04
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-06
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run maplibre map workspace against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-06 — AOI draw/edit and validation UX

Field	Implementation specification
Priority	P0
Type	Geo UX
Owner	P5
Reviewer	P1
Cross-service contact	P2
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement aoi draw/edit and validation ux as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for aoi draw/edit and validation ux. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_aoi_draw_edit_and_validation_ux_valid()', 'test_aoi_draw_edit_and_validation_ux_invalid_input()']
Integration tests	['test_p5_06_service_boundary()']
Contract tests	['test_p5_06_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, aoi draw/edit and validation ux completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-07
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run aoi draw/edit and validation ux against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-07 — Observation and before/after comparison

Field	Implementation specification
Priority	P0
Type	Geo UX
Owner	P5
Reviewer	P1
Cross-service contact	P1
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement observation and before/after comparison as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for observation and before/after comparison. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_observation_and_before_after_comparison_valid()', 'test_observation_and_before_after_comparison_invalid_input()']
Integration tests	['test_p5_07_service_boundary()']
Contract tests	['test_p5_07_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, observation and before/after comparison completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-06,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-08
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run observation and before/after comparison against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-08 — Change polygons, masks and confidence visualization

Field	Implementation specification
Priority	P0
Type	Geo UX
Owner	P5
Reviewer	P1
Cross-service contact	P4
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement change polygons, masks and confidence visualization as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for change polygons, masks and confidence visualization. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_change_polygons_masks_and_confidence_visualization_valid()', 'test_change_polygons_masks_and_confidence_visualization_invalid_input()']
Integration tests	['test_p5_08_service_boundary()']
Contract tests	['test_p5_08_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, change polygons, masks and confidence visualization completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-07,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-09
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run change polygons, masks and confidence visualization against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-09 — WHY? evidence panel

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P5
Reviewer	P1
Cross-service contact	P2
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement why? evidence panel as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for why? evidence panel. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_why_evidence_panel_valid()', 'test_why_evidence_panel_invalid_input()']
Integration tests	['test_p5_09_service_boundary()']
Contract tests	['test_p5_09_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, why? evidence panel completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-08,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-10
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run why? evidence panel against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-10 — Sensor arbitration and disagreement UI

Field	Implementation specification
Priority	P1
Type	Feature
Owner	P5
Reviewer	P1
Cross-service contact	P1
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement sensor arbitration and disagreement ui as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for sensor arbitration and disagreement ui. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_sensor_arbitration_and_disagreement_ui_valid()', 'test_sensor_arbitration_and_disagreement_ui_invalid_input()']
Integration tests	['test_p5_10_service_boundary()']
Contract tests	['test_p5_10_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, sensor arbitration and disagreement ui completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-09,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-11
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run sensor arbitration and disagreement ui against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-11 — Confidence/uncertainty and human-in-loop UX

Field	Implementation specification
Priority	P0
Type	Safety UX
Owner	P5
Reviewer	P1
Cross-service contact	P4
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement confidence/uncertainty and human-in-loop ux as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for confidence/uncertainty and human-in-loop ux. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_confidence_uncertainty_and_human_in_loop_ux_valid()', 'test_confidence_uncertainty_and_human_in_loop_ux_invalid_input()']
Integration tests	['test_p5_11_service_boundary()']
Contract tests	['test_p5_11_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, confidence/uncertainty and human-in-loop ux completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-10
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-12
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run confidence/uncertainty and human-in-loop ux against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-12 — Mission memory and temporal history

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P5
Reviewer	P1
Cross-service contact	P2
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement mission memory and temporal history as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for mission memory and temporal history. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_mission_memory_and_temporal_history_valid()', 'test_mission_memory_and_temporal_history_invalid_input()']
Integration tests	['test_p5_12_service_boundary()']
Contract tests	['test_p5_12_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, mission memory and temporal history completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-11
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-13
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run mission memory and temporal history against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-13 — Persistent monitoring mission screen

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P5
Reviewer	P1
Cross-service contact	P1
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement persistent monitoring mission screen as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for persistent monitoring mission screen. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_persistent_monitoring_mission_screen_valid()', 'test_persistent_monitoring_mission_screen_invalid_input()']
Integration tests	['test_p5_13_service_boundary()']
Contract tests	['test_p5_13_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, persistent monitoring mission screen completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-12
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-14
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run persistent monitoring mission screen against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-14 — Report generation and report view

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P5
Reviewer	P1
Cross-service contact	P4
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement report generation and report view as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for report generation and report view. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_report_generation_and_report_view_valid()', 'test_report_generation_and_report_view_invalid_input()']
Integration tests	['test_p5_14_service_boundary()']
Contract tests	['test_p5_14_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, report generation and report view completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-13
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-15
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run report generation and report view against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-15 — Admin, security and accessibility UX

Field	Implementation specification
Priority	P1
Type	Security/UX
Owner	P5
Reviewer	P1
Cross-service contact	P2
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement admin, security and accessibility ux as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for admin, security and accessibility ux. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_admin_security_and_accessibility_ux_valid()', 'test_admin_security_and_accessibility_ux_invalid_input()']
Integration tests	['test_p5_15_service_boundary()']
Contract tests	['test_p5_15_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, admin, security and accessibility ux completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-14,P4-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-16
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run admin, security and accessibility ux against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-16 — Frontend performance and error boundaries

Field	Implementation specification
Priority	P1
Type	Performance
Owner	P5
Reviewer	P1
Cross-service contact	P1
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement frontend performance and error boundaries as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for frontend performance and error boundaries. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_frontend_performance_and_error_boundaries_valid()', 'test_frontend_performance_and_error_boundaries_invalid_input()']
Integration tests	['test_p5_16_service_boundary()']
Contract tests	['test_p5_16_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, frontend performance and error boundaries completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-15, P4-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P5-17
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run frontend performance and error boundaries against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P5-17 — E2E UX hardening and deterministic demo mode

Field	Implementation specification
Priority	P0
Type	Release
Owner	P5
Reviewer	P1
Cross-service contact	P4
Service	Web / Map UX
Folders	apps/web/app/; apps/web/components/; apps/web/lib/api/; apps/web/lib/geo/; apps/web/e2e/; packages/contracts/generated/ts/
Objective	Implement e2e ux hardening and deterministic demo mode as a production-shaped module inside the P5 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for e2e ux hardening and deterministic demo mode. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	apps/web/app/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	Gateway-only browser calls: missions, runs, evidence, observations, tiles, reports, monitoring, WebSocket
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_e2e_ux_hardening_and_deterministic_demo_mode_valid()', 'test_e2e_ux_hardening_and_deterministic_demo_mode_invalid_input()']
Integration tests	['test_p5_17_service_boundary()']
Contract tests	['test_p5_17_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, e2e ux hardening and deterministic demo mode completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P5-16,P4-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	Release/integration gate
Parallelizable with	P4 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P5. Consumer/contact: P4. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run e2e ux hardening and deterministic demo mode against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

7. Test Plan

The issue-level test names below are the minimum required suite. Expand edge cases discovered during implementation.

P5-01 — Next.js application shell and design system

- test_next_js_application_shell_and_design_system_valid()
- test_next_js_application_shell_and_design_system_invalid_input()
- test_p5_01_service_boundary()
- test_p5_01_schema_compatibility()

P5-02 — Generated API client integration

- test_generated_api_client_integration_valid()
- test_generated_api_client_integration_invalid_input()
- test_p5_02_service_boundary()
- test_p5_02_schema_compatibility()

P5-03 — Mission console query and planning UX

- test_mission_console_query_and_planning_ux_valid()
- test_mission_console_query_and_planning_ux_invalid_input()
- test_p5_03_service_boundary()
- test_p5_03_schema_compatibility()

P5-04 — Mission status and run timeline

- test_mission_status_and_run_timeline_valid()
- test_mission_status_and_run_timeline_invalid_input()
- test_p5_04_service_boundary()
- test_p5_04_schema_compatibility()

P5-05 — MapLibre map workspace

- test_maplibre_map_workspace_valid()
- test_maplibre_map_workspace_invalid_input()
- test_p5_05_service_boundary()
- test_p5_05_schema_compatibility()

P5-06 — AOI draw/edit and validation UX

- test_aoi_draw_edit_and_validation_ux_valid()
- test_aoi_draw_edit_and_validation_ux_invalid_input()
- test_p5_06_service_boundary()
- test_p5_06_schema_compatibility()

P5-07 — Observation and before/after comparison

- test_observation_and_before_after_comparison_valid()
- test_observation_and_before_after_comparison_invalid_input()
- test_p5_07_service_boundary()
- test_p5_07_schema_compatibility()

P5-08 — Change polygons, masks and confidence visualization

- test_change_polygons_masks_and_confidence_visualization_valid()
- test_change_polygons_masks_and_confidence_visualization_invalid_input()
- test_p5_08_service_boundary()
- test_p5_08_schema_compatibility()

P5-09 — WHY? evidence panel

- test_why_evidence_panel_valid()
- test_why_evidence_panel_invalid_input()
- test_p5_09_service_boundary()
- test_p5_09_schema_compatibility()

P5-10 — Sensor arbitration and disagreement UI

- test_sensor_arbitration_and_disagreement_ui_valid()
- test_sensor_arbitration_and_disagreement_ui_invalid_input()
- test_p5_10_service_boundary()
- test_p5_10_schema_compatibility()

P5-11 — Confidence/uncertainty and human-in-loop UX

- test_confidence_uncertainty_and_human_in_loop_ux_valid()
- test_confidence_uncertainty_and_human_in_loop_ux_invalid_input()
- test_p5_11_service_boundary()
- test_p5_11_schema_compatibility()

P5-12 — Mission memory and temporal history

- test_mission_memory_and_temporal_history_valid()
- test_mission_memory_and_temporal_history_invalid_input()
- test_p5_12_service_boundary()
- test_p5_12_schema_compatibility()

P5-13 — Persistent monitoring mission screen

- test_persistent_monitoring_mission_screen_valid()
- test_persistent_monitoring_mission_screen_invalid_input()
- test_p5_13_service_boundary()
- test_p5_13_schema_compatibility()

P5-14 — Report generation and report view

- test_report_generation_and_report_view_valid()
- test_report_generation_and_report_view_invalid_input()
- test_p5_14_service_boundary()
- test_p5_14_schema_compatibility()

P5-15 — Admin, security and accessibility UX

- test_admin_security_and_accessibility_ux_valid()
- test_admin_security_and_accessibility_ux_invalid_input()
- test_p5_15_service_boundary()
- test_p5_15_schema_compatibility()

P5-16 — Frontend performance and error boundaries

- test_frontend_performance_and_error_boundaries_valid()
- test_frontend_performance_and_error_boundaries_invalid_input()
- test_p5_16_service_boundary()
- test_p5_16_schema_compatibility()

P5-17 — E2E UX hardening and deterministic demo mode

- test_e2e_ux_hardening_and_deterministic_demo_mode_valid()
- test_e2e_ux_hardening_and_deterministic_demo_mode_invalid_input()
- test_p5_17_service_boundary()
- test_p5_17_schema_compatibility()

8. Security Implementation

Control	Implementation owner	Required behavior
Authentication	P5	Implement/test where applicable; P6 supplies regression gate.
RBAC + tenant isolation	P5	Implement/test where applicable; P6 supplies regression gate.
Input/schema validation	P5	Implement/test where applicable; P6 supplies regression gate.
GeoJSON/raster validation	P5	Implement/test where applicable; P6 supplies regression gate.
SSRF/provider allowlist	P5	Implement/test where applicable; P6 supplies regression gate.
Prompt injection	P5	Implement/test where applicable; P6 supplies regression gate.
AI tool permissions	P5	Implement/test where applicable; P6 supplies regression gate.
Rate/request limits	P5	Implement/test where applicable; P6 supplies regression gate.
Secrets/TLS	P5	Implement/test where applicable; P6 supplies regression gate.
Audit logging	P5	Implement/test where applicable; P6 supplies regression gate.
Dependency/container scanning	P5	Implement/test where applicable; P6 supplies regression gate.

9. Observability

Required identifiers: trace_id, mission_id, run_id, job_id, organization_id, model_version, dataset_id. Metrics should expose latency, queue time, errors and user-visible mission state. Role metrics: web_interaction_latency_ms, api_error_total, map_layer_load_ms, ws_reconnect_total

10. Failure Modes

Failure	Detection	Recovery/fallback	User behavior
Bhoonidhi unavailable	timeout/5xx	bounded retry → pinned permitted fixture	show fallback clearly
Inference timeout/OOM	job timeout/worker exit	bounded retry → deterministic fixture for demo	show explicit degraded/failure state
Redis restart	connection/requeue failure	idempotent requeue/reconciliation	mission remains recoverable
Invalid AOI/raster	validator rejection	no expensive processing	show actionable validation
Low confidence	confidence gate	clarify/acquire/explicit uncertainty	never confident prose
Tenant violation	RBAC denial	no retry/no leakage	403/404

11. Implementation Order

Foundation → core domain → cross-service integration → differentiation → hardening/release. Earlier issues unlock later ones, but frozen contracts and fixtures allow parallel implementation. Do not serialize on live dependencies. Use mocks whenever the producer is unfinished.

12. Daily Execution Plan — Sep 11–20

Date	Issues	Expected output	Dependency	Integration checkpoint	Tests/demo
Sep 11	P5-01, P5-02	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 12	P5-02, P5-03	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 13	P5-03, P5-04	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 14	P5-04, P5-05	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 15	P5-05, P5-06	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 16	P5-06, P5-07	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 17	P5-07, P5-08	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 18	P5-08, P5-09	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 19	P5-09, P5-10	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 20	P5-10, P5-11	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state

13. Definition of Ready / Done

Field	Implementation specification
DoR	Requirements clear; owner/reviewer; contract; dependencies; acceptance; tests; demo result defined.
DoD	Merged; unit/contract/integration tests; security; telemetry; docs/schema; DEV/DEMO; acceptance demonstrated.

14. Cross-Team Communication

- P1 is architecture/contract authority.
- P6 is CI/DEV/DEMO/release authority.
- P2↔P3: sensor arbitration, inference and evidence.
- P2↔P4: observation selection, data quality and acquisition.
- P3↔P4: AssetRef, CRS/resolution and preprocessing.
- P4↔P5: tiles, masks, GeoJSON and map performance.
- P5↔P1: generated client and browser DTOs.
- Escalate blockers after 30–60 minutes; use contract mocks/fixtures instead of waiting.

15. Prompt to Give to the Coding Agent

You are P5 implementation agent. Own Next.js map-first command center. Browser calls Gateway only and consumes generated TypeScript contracts. Build mission console, map/AOI, before/after, change/confidence, WHY evidence, sensor disagreement, timeline, monitoring and report UX. Backend is authoritative for permissions/state; never fake progress. Do not store long-lived tokens in localStorage or render arbitrary HTML. Add component/accessibility/Playwright/contract tests.

16. Role Release Checklist

- ☐ All P0 issues merged or explicitly waived.
- ☐ Unit + contract + relevant integration tests green.
- ☐ Security controls and negative tests green.
- ☐ Trace/metrics/logging present.
- ☐ DEV/DEMO verified.
- ☐ Failure behavior demonstrated.
- ☐ Fixtures/model versions pinned where applicable.
- ☐ Documentation/ADR updated.
- ☐ Role-specific flagship demo path works.